Beyond Recognition Accuracy: Efficient CPU Streaming ASR for Dysarthric Speech

Sanjid Hasan[1] **Zaina Rahman**[2] **Suhita Islam Aurthi**[2]
Md Rashaduzzaman Rakib[2] Sadia Jahan Turabe[2] Md. Wasif Rahman[2]
[1]Dept. of Electrical and Electronic Engineering [2]Dept. of Computer Science and Engineering
Khulna University of Engineering and Technology, Khulna 9203, Bangladesh
{hasan2203090,rahman2207021,aurthi2207052,
rakib2307038,turabe2307065,rahman2307101}@stud.kuet.ac.bd

Abstract

Real-time dysarthric speech recognition is difficult because articulation varies widely across speakers while streaming systems must run under strict CPU latency limits. We present our Track 2 (Streaming ASR) system for the Speech Accessibility Project Challenge 2 (SAPC2). We fine-tuned a 66.1M-parameter causal Zipformer transducer on the SAPC2 training set, supplemented with TORGO recordings and 41,850 synthetic utterances from Qwen and Kokoro TTS. Training used randomized causal chunk sizes, SpecAugment, and acoustic perturbations. We also expanded the vocabulary from 500 to 510 tokens to emit digits directly while keeping the original token IDs intact. For CPU deployment, we exported the model to mixed-precision INT8 ONNX and streamed 100-ms audio chunks through `sherpa-onnx`. On the official Test1 leaderboard, our greedy model reached 22.18% CER and 31.73% WER—beating the 34.59% CER baseline—with a 1.38 s median stable first-token latency and 117 ms last-token delay. On a 501-utterance local set, 4-path beam search further lowered WER to 19.47% (12.59% CER). Perceptual annotations of over 437 samples show that recognition errors track perceived speech clarity (19.30% WER on clear vs. 57.28% on unclear speech) at least as strongly as medical etiology. Finally, we introduce Cadrate ASR, a 2.70M-parameter dual-rate CNN–GRU prototype that cuts CPU compute time below Zipformer, highlighting the trade-off between raw inference speed and recognition accuracy.

Index Terms: Dysarthric Speech, Streaming ASR, Zipformer Transducer, INT8 Quantization, Perceptual Analysis

1 Introduction

Dysarthria is a motor speech disorder linked to neurological conditions such as Parkinson's disease, cerebral palsy, ALS, Down syndrome, and stroke. It disrupts vocal control and articulation, producing slurred, slow, or irregular speech that varies across speakers and even within a single utterance [16, 8]. Standard ASR models trained on typical voices struggle with this variability. Streaming ASR makes the problem even harder. A streaming model must emit text incrementally from short audio chunks without seeing future context.

Earlier Speech Accessibility Project (SAP) systems relied on heavy offline models like Parakeet-TDT 1.1B [20] and Whisper large-v3 [23, 21] running on GPUs [25]. The Speech Accessibility Project Challenge 2 (SAPC2) shifts the focus to real-time accessibility [19]. In Track 2, systems process incoming 100-ms audio chunks strictly on CPU hardware [18]. Models are judged on both accuracy—character error rate (CER) and word error rate (WER)—and responsiveness, measured

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

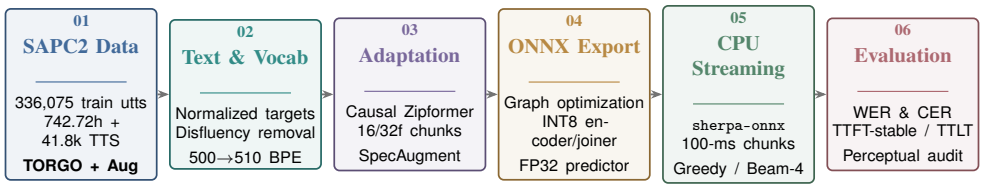


Figure 1: Overview of our SAPC2 Track 2 streaming ASR pipeline, spanning domain-specific data curation, digit-extended vocabulary transplantation, causal Zipformer fine-tuning, mixed-precision INT8 ONNX export, and multi-dimensional accuracy–latency evaluation.

via stable time to first token (TTFT-stable) and time to last token (TTLT). How compact, causal transducers navigate this accuracy–latency trade-off remains an open question.

To address this, we adapt a pretrained 66.1M-parameter causal Zipformer transducer [24] on the full 742.72-hour SAPC2 training set, supplemented with TORGO [16] and 41,850 synthetic TTS utterances. We train with randomized causal chunk sizes, SpecAugment [14], and a transplanted 510-token vocabulary [7] that adds ten digit tokens (‘0‘–‘9‘) without overwriting pretrained weights. Deployed on CPU as a mixed-precision INT8 ONNX model via `sherpa-onnx` [10], our greedy submission hits 22.18% CER and 31.73% WER on the official Test1 leaderboard. This beats the 34.59% CER challenge baseline while maintaining a 1.38 s median TTFT-stable and 117 ms median TTLT.

Beyond our leaderboard submission, we present three key findings:

- **Decoding vs. Latency:** Without retraining, switching from greedy search to 4-path modified beam search drops local WER from 21.74% to 19.47% (12.59% CER), with only a minor increase in total CPU decode time.
- **Perceptual Error Stratification:** Comparing Zipformer against FastConformer [15], Nemotron INT4, and Moonshine v2 [12] across 437+ manually annotated samples reveals that WER jumps from 19.30% on clear dysarthric speech to 57.28% on unclear speech—showing that perceived vocal clarity drives ASR errors just as strongly as clinical etiology.
- **Extreme Efficiency Limits:** We build **Cadrate ASR**, a 2.70M-parameter dual-rate causal CNN–GRU prototype trained from scratch. While it beats Zipformer in CPU real-time factor and P95 chunk latency, its lower accuracy proves that ultra-fast architectures still need large-scale pretraining to handle disordered speech.

2 Related Work

Dysarthric Speech Recognition and SAP Challenges. Early dysarthric ASR research focused on small-scale corpora such as UA-Speech, TORGO [16], and EasyCall, which contain isolated words or short commands from few speakers. The Speech Accessibility Project (SAP) [8] enabled large-scale, speaker-independent modeling and established the 2025 SAP Challenge [25]. Top-performing 2025 systems adapted large offline backbones: Takahashi et al. [20] fine-tuned Parakeet-TDT 1.1B with speed perturbation and wide-beam decoding, while Wang et al. [23] applied self-training to Whisper large-v3. Other competitive approaches explored pathology-aware speech encoding [2], controllable LLM-prompted synthetic speech [22], model merging [3], and curriculum-based AdaLoRA fine-tuning [21]. However, all of these systems relied on non-causal full-utterance attention and GPU inference, rendering them incompatible with SAPC2 Track 2’s strict CPU streaming constraints [19].

Streaming Architectures and On-Device Deployment. Incremental speech recognition relies on causal transducers such as RNN-T [4] and efficient encoders including Zipformer [24], FastConformer [15], and ergodic streaming models like Moonshine v2 [12]. Deploying these models on CPU requires aggressive quantization and runtime optimization [1, 10]. Crucially, raw computational speed (e.g., real-time factor) does not fully capture user-perceived responsiveness, which depends heavily on token emission delay, partial hypothesis stability, and endpointing [17]. We therefore evaluate both

stable first-token latency (TTFT-stable) and finalization delay (TTLT) across quantized streaming candidates.

Data Augmentation and Vocabulary Adaptation. Spectral masking via SpecAugment [14], synthetic TTS data generation [22], and multi-stage acoustic perturbations [6] improve ASR robustness under domain shift. Simultaneously, subword vocabulary adaptation techniques such as tokenizer transplantation [7] allow models to emit domain-specific tokens without destroying pretrained weights. We combine conservative digit-token transplantation with multi-source augmentation to stabilize streaming transducer decoding on dysarthric speech.

3 Dataset Preparation and Augmentation

Table 1: Summary of SAPC2 corpus splits, speaker etiology breakdown, and augmentation sources.

Dataset Split / Source	Utterances	Hours	Speakers	Key Characteristics & Distribution
SAPC2 Train [8, 19]	336,075	742.72	875	PD: 311, CP: 167, ALS: 167, DS: 145, Stroke: 85
SAPC2 Dev [19]	47,929	106.62	124	0% speaker overlap; 20.02% text prompt overlap
Synthetic TTS (Qwen + Kokoro)	41,850	—	—	Timing, channel, and volume perturbations
TORGO + 6-Stage Acoustic Aug.	Suppl.	—	—	Noise, $0.9\times$ stretch, $+2$ st pitch, 4 kHz LPF, echo, reverb

Corpus Dynamics and Normalization. Table 1 summarizes our training and evaluation data. The SAPC2 Train and Dev splits have zero speaker overlap, enforcing strict zero-shot speaker evaluation [8, 19]. Most recordings (82%) are under 10 s; ALS speakers average the longest utterances (8.97 s) due to slower speech, while Down syndrome speakers average the shortest (6.73 s). Although every audio recording is unique, 68% of training files share repeated voice-assistant commands (e.g., *"Start listening."*, *"Hey Google."*, *"Alexa."*, each spoken over 2,100 times across 830+ speakers). We trained on the disfluency-free normalized transcripts, yielding 88,694 unique text targets.

Synthetic and Acoustic Augmentation. To boost lexical and acoustic diversity, we added TORGO dysarthric speech [16] and 41,850 synthetic clips from Qwen-TTS and Kokoro-TTS. Inspired by extreme acoustic hardening [6], we also built a 6-stage distortion pipeline—cascading Gaussian noise ($\sigma = 0.02$), $0.9\times$ time stretch, $+2$-semitone pitch shift, 4 kHz Butterworth low-pass filtering, single-tap echo, and five-tap reverb—to separate channel degradation from dysarthric articulation (see Appendix C).

4 Streaming ASR Architectures and Deployment

4.1 Primary System: Causal Zipformer and Vocabulary Transplantation

We build our primary Track 2 submission on a pretrained streaming Zipformer RNN-T model ($\sim$66.1M parameters) trained via the Icefall framework [24, 9]. Its multi-rate causal encoder processes 80-dimensional log-Mel filterbank features incrementally without future lookahead, paired with a stateless prediction network and pruned transducer loss.

Because normalized SAPC2 transcripts contain numerals that the base 500-token BPE vocabulary decomposes sub-optimally, we applied conservative vocabulary expansion [7] from 500 to 510 tokens by appending the ten ASCII digits (‘0‘–‘9‘). To avoid catastrophic forgetting, the original 500 embedding and joiner projection rows were preserved intact, while the 10 new token rows were initialized with scaled Gaussian noise and zero bias.

We fine-tuned the model on a single NVIDIA RTX 4070 GPU (12 GB VRAM) using FP16 mixed precision, a base learning rate of 3×10^{-5}, duration-based dynamic batching (≤ 120 s per batch), and SpecAugment [14]. To match the 100-ms streaming harness [18], training randomized causal chunk sizes between 16 and 32 frames with left-context lengths of 128 or 256 frames. Validation loss and streaming WER were tracked across 19 epochs, with Epoch 19 (E19) selected as our primary checkpoint. For CPU execution, we exported the E19 checkpoint to ONNX, applied INT8 dynamic quantization to the encoder and joiner while keeping the lightweight predictor in FP32, and deployed via `sherpa-onnx` [10]. We evaluate both greedy decoding (official submission) and 4-path modified beam search with 0.96 s–1.28 s tail padding.

Table 2: Architectural and deployment comparison between our adapted Zipformer system and our proposed Cadrate ASR prototype.

Specification	Adapted Zipformer (Primary)	Cadrate ASR (Prototype)
Encoder Backbone	Multi-rate Causal Transformer (6 stacks)	Dual-rate Causal Depthwise CNN–GRU
Temporal Resolutions	50, 25, 12.5, 6.25, 12.5, 25 Hz	25 Hz (fast acoustic) & 12.5 Hz (slow context)
Cross-Rate Integration	Downsampling, upsampling, bypasses	Causal gated additive channel fusion
Decoding Objective	Pruned RNN-T (Greedy / Modified Beam-4)	Framewise CTC (Greedy collapse)
Vocabulary & Precision	510 BPE (INT8 encoder/joiner, FP32 pred.)	510 BPE (FP32 / INT8 streaming state)
Trainable Parameters	$\sim$66.1M (pretrained + fine-tuned)	$\sim$2.70M (trained from scratch)

Table 3: Comparison of offline baselines, zero-shot streaming candidates, and our adapted Zipformer configurations across the SAPC2 official Test1 leaderboard and local development splits.

System / Configuration	Evaluation Split	WER (%)	CER (%)	TTFT-stable	TTLT / Speed
Official SAPC2 Track 2 Leaderboard (CPU Streaming, 100-ms chunks)					
SAPC2 Track 2 Challenge Baseline	Official Test1	52.77	34.59	—	367 ms
Ours: Zipformer INT8 (Greedy)	**Official Test1**	**31.73**	**22.18**	**1.38 s**	**117 ms**
Offline vs. Streaming Paradigms & Local Development Evaluations (Dev-501 / Dev-100)					
Whisper Large-v3 (Zero-shot Offline)	Dev (Offline)	36.56	15.34	N/A	RTF $>$ 1.0 (CPU)
Parakeet RNN-T (Fine-tuned Offline)	Dev (Offline)	22.22	9.67	N/A	RTF $>$ 1.0 (CPU)
Zipformer Pretrained (Zero-shot)	Dev-501 (Stream)	58.20	34.91	N/A	—
Zipformer Fine-tuned (Val Baseline)	Dev-501 (Stream)	27.48	18.90	N/A	—
Zipformer E19 INT8 (Greedy)	Dev-501 / 100	21.74 / 22.02	— / 14.70	$\sim$395 ms†	140 s total
Zipformer E19 INT8 (Beam-4)	**Dev-501 / 100**	**19.47**	**12.59**	**$\sim$395 ms†**	**157 s total**
Zipformer E19 INT8 (Beam-4, Pad 1.28s)	**Dev-100**	**19.44**	**12.58**	—	**277 s total**
Cadrate ASR 2.70M (From Scratch)	Local Val	High WER	—	Ultra-low	Lowest CPU RTF

†Denotes median local Time-to-First-Token (TTFT). Complete 15-configuration decoding ablations and 57-utterance Moonshine v2 benchmarks are reported in Appendix B.

4.2 Proposed Prototype: Cadrate ASR

To investigate the lower bound of CPU compute and state memory, we developed **Cadrate ASR**, a compact 2.70M-parameter causal dual-rate architecture trained from scratch. Cadrate ASR stacks 4 consecutive 100-Hz filterbank frames into a 25-Hz fast acoustic pathway (256 channels, 8 gated depthwise dilated convolutional blocks with kernel size $k = 7$ and dilations $\{1, 2, 4, 8\}$) alongside a 12.5-Hz slow contextual pathway (192 channels, 4 causal convolutional blocks followed by a unidirectional GRU). A channel-wise sigmoid-gated additive fusion module injects the slow recurrent context into the fast pathway while strictly preserving causal alignment, followed by a framewise CTC head. Unlike Zipformer's attention caches and RNN-T decoder, Cadrate ASR maintains a fixed 249 KiB neural state (63,744 FP32 values) across chunks and uses greedy CTC decoding. Table 2 summarizes both systems; full mathematical formulations and complexity proofs are provided in Appendix A.

5 Experimental Results and Analysis

5.1 Streaming vs. Offline Reality and Official Leaderboard Performance

As shown in Table 3, heavy offline models such as Whisper Large-v3 (36.56% WER) and our fine-tuned Parakeet RNN-T (22.22% WER) perform well with full-utterance context, but fail in the 100-ms CPU streaming harness with Real-Time Factors (RTF) above 1.0. Conversely, a zero-shot streaming Zipformer yields 58.20% WER on dysarthric speech prior to domain adaptation.

Domain fine-tuning and digit-token transplantation lowered greedy streaming WER to 27.48% on validation and **31.73% WER / 22.18% CER** on the official SAPC2 Test1 leaderboard—a **12.41% absolute CER reduction** over the 34.59% challenge baseline—with a median TTFT-stable of 1.38 s and median TTLT of 117 ms (Figure 2). On our 501-utterance local set, switching from greedy search (21.74% WER) to 4-path modified beam search in mixed-precision INT8 further dropped WER to **19.47%** (12.59% CER) with a median local TTFT of $\sim$395 ms and only a 17 s increase in total

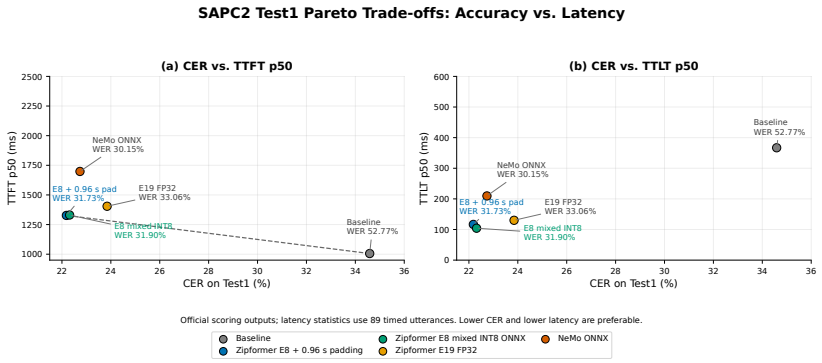


Figure 2: Accuracy–latency trade-offs on SAPC2 Test1 comparing character error rate (CER) against median time to first token (TTFT) and median time to last token (TTLT) across 89 timed utterances. Lower values are preferable.

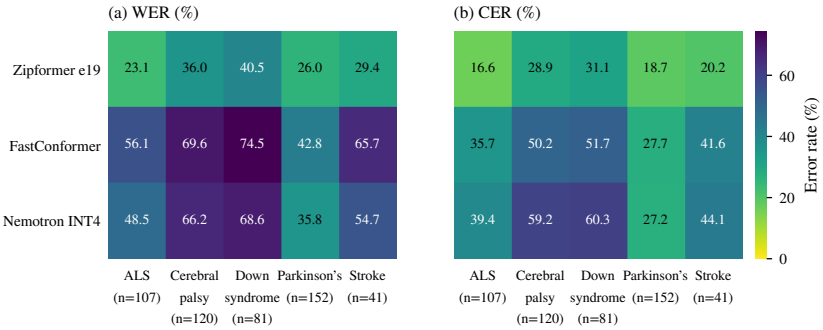


Figure 3: Heatmap of ASR error rates (WER/CER) across clinical etiologies, illustrating strong performance on Parkinson's Disease and ALS versus elevated error rates on Cerebral Palsy and Down Syndrome.

decode time (157 s vs. 140 s; full 15-run sweep in Appendix B.2). Meanwhile, our 2.70M-parameter Cadrate ASR prototype achieved lower CPU RTF and P95 chunk latency than Zipformer, but did not generalize well to unseen speakers, confirming that low compute must be paired with pretrained acoustic representations.

5.2 Performance Across Clinical Etiologies and Perceptual Characteristics

Breaking errors down by clinical etiology (Figure 3) shows that Zipformer E19 performed best on Parkinson's Disease and ALS (CERs consistently below 20%), where acoustic patterns remain relatively stable. In contrast, error rates rose to 30–47% on Cerebral Palsy and Down Syndrome due to severe articulatory and timing variability.

To look past broad clinical labels, we manually audited 437+ recordings from the 501-utterance evaluation set across perceptual dimensions of clarity, articulation, fluency, and speech rate (protocol in Appendix D). Table 4 compares Zipformer E19 against FastConformer [15] and Nemotron INT4 across the eight main perceptual categories ($n \geq 16$; rare categories with $n \leq 7$ are in Appendix D). Zipformer E19 beats both baselines by 15–44% absolute WER in every category. Crucially, errors track perceived speech clarity closely: Zipformer WER jumps from **19.30%** on *Clear* ($n = 152$) and **21.02%** on *Slow* ($n = 21$) speech to **33.56–35.30%** on *Imprecise*, *Stammered*, and *Slurred* speech, reaching **57.28%** (44.52% CER) on *Unclear* speech ($n = 46$).

Table 4: ASR performance across manually annotated perceptual speech characteristics ($n \geq 16$) on the 501-utterance evaluation cohort. Lower WER and CER (%) indicate better recognition.

Perceptual Characteristic	n	Zipformer E19 (Ours)		FastConformer		Nemotron INT4	
		WER	CER	WER	CER	WER	CER
Clear	152	**19.30**	**13.78**	40.15	25.32	33.03	24.70
Slow	21	**21.02**	**14.36**	42.05	25.83	28.30	23.46
Muffled	30	**23.08**	**14.64**	40.00	24.77	38.46	26.56
Drowsy	25	**28.68**	**21.18**	67.55	45.43	49.81	39.61
Imprecise articulation	19	**33.56**	**25.98**	77.18	51.08	77.85	68.69
Stammered	16	**33.95**	**26.18**	72.84	49.34	70.37	60.13
Slurred	48	**35.30**	**26.77**	68.05	47.24	62.30	54.15
Unclear	46	**57.28**	**44.52**	89.74	65.93	90.40	81.01

6 Discussion

Our experiments highlight four practical lessons for building real-time dysarthric ASR. First, **domain adaptation matters more than model size**. Fine-tuning a compact 66.1M-parameter causal Zipformer with randomized chunk sizes and digit-token transplantation beat every unadapted streaming baseline we tested, including FastConformer, Nemotron INT4, and Moonshine v2. Architecture alone does not solve disordered speech; direct exposure to dysarthric acoustics is essential.

Second, **accuracy and output stability pull in opposite directions**. Switching from greedy decoding to 4-path beam search with 1.28 s tail padding cut local WER by over 2.5% absolute without retraining. However, beam search revises partial hypotheses as new chunks arrive, widening the gap between the first visible word and the first stable word (TTFT-stable) [17]. For someone using a speech interface, a slightly slower transcript that stays put is often more helpful than an instant transcript that keeps flickering.

Third, **raw CPU speed cannot replace pretraining**. Mixed-precision INT8 ONNX export allowed our Zipformer to stream smoothly on CPU. While our 2.70M-parameter Cadrate ASR prototype pushed CPU latency and memory state even lower (249 KiB), training from scratch left it unable to generalize to unseen dysarthric speakers. Low compute cost is only useful once recognition accuracy crosses a practical threshold.

Finally, **clinical labels hide wide variations in speech clarity**. Our manual audit of 437+ recordings showed that WER tracks perceived vocal clarity (19.30% on clear vs. 57.28% on unclear speech) at least as closely as medical diagnosis. Standardizing perceptual benchmarks alongside clinical etiologies can help pinpoint where streaming decoders break down.

7 Conclusion and Future Directions

We presented a domain-adapted causal Zipformer for SAPC2 Track 2, combining multi-chunk fine-tuning, digit-token extension, and synthetic speech augmentation. Our submission achieved 22.18% CER and 31.73% WER on official Test1, substantially outperforming the 34.59% CER baseline, with median TTFT-stable of 1.38 s and TTLT of 117 ms. A separate 100-recording experiment further reduced WER to 19.47% using beam-4 decoding. Our perceptual analysis revealed persistent challenges with unclear, slurred, and imprecisely articulated speech, while Cadrate ASR highlighted the trade-off between computational efficiency and recognition accuracy.

Future work will focus on improving robustness across diverse speech characteristics, reducing stable-output latency, and developing lightweight architectures that generalize reliably to unseen speakers. Ultimately, accessible streaming ASR must deliver not only fast predictions, but accurate and stable recognition for people with diverse speech impairments.

References

[1] Nenad Banfic, David Fan, Kunal Vaishnavi, Sam Kemp, Sunghoon Choi, Rui Ren, Sayan Shaw, and Meng Tang. Pushing the limits of on-device streaming ASR: A compact, high-accuracy english model for low-latency inference. *arXiv preprint arXiv:2604.14493*, 2026.

[2] Ilja Baumann, Dominik Wagner, Korbinian Riedhammer, and Tobias Bocklet. Pathology-aware speech encoding and data augmentation for dysarthric speech recognition. In *Proceedings of Interspeech 2025*, pages 3289–3293, 2025.

[3] Alexandre Ducorroy and Rachid Riad. Robust fine-tuning of speech recognition models via model merging: application to disordered speech. In *Proceedings of Interspeech 2025*, pages 3279–3283, 2025.

[4] Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

[5] Wei Han, Zhengdong Zhang, Yu Zhang, Jiahui Yu, Chung-Cheng Chiu, James Qin, Anmol Gulati, Ruoming Pang, and Yonghui Wu. ContextNet: Improving convolutional neural networks for automatic speech recognition with global context. In *Proceedings of Interspeech*, pages 3610–3614, 2020.

[6] Sanjid Hasan, Risalat Labib, A H M Fuad, and Bayazid Hasan. Make it hard to hear, easy to learn: Long-form bengali ASR and speaker diarization via extreme augmentation and perfect alignment. *arXiv preprint arXiv:2602.23070*, 2026.

[7] Sanjid Hasan and Md. Abdur Rahman. Tokenizer transplantation: Mitigating autoregressive collapse in edge-efficient bengali ASR. *arXiv preprint arXiv:2607.09598*, 2026. Accepted at MusIML Workshop, ICML 2026.

[8] Mark Hasegawa-Johnson, Xiuwen Zheng, Heejin Kim, Clarion Mendes, Meg Dickinson, Erik Hege, Chris Zwilling, Marie Moore Channell, Laura Mattie, Heather Hodges, Lorraine Ramig, Mary Bellard, Mike Shebanek, Leda Sarı, Kaustubh Kalgaonkar, David Frerichs, Jeffrey P. Bigham, Leah Findlater, Colin Lea, Sarah Herrlinger, Peter Korn, Shadi Abou-Zahra, Rus Heywood, Katrin Tomanek, and Bob MacDonald. Community-supported shared infrastructure in support of speech accessibility. *Journal of Speech, Language, and Hearing Research*, 67(11):4162–4175, 2024.

[9] k2-fsa contributors. icefall: Speech recognition recipes. GitHub repository. Accessed November 2, 2026.

[10] k2-fsa contributors. sherpa-onnx. Software and documentation. Accessed November 2, 2026.

[11] Samuel Kriman, Stanislav Beliaev, Boris Ginsburg, Jocelyn Huang, Oleksii Kuchaiev, Vitaly Lavrukhin, Ryan Leary, Jason Li, and Yang Zhang. QuartzNet: Deep automatic speech recognition with 1d time-channel separable convolutions. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6124–6128, 2020.

[12] Manjunath Kudlur, Evan King, James Wang, and Pete Warden. Moonshine v2: Ergodic streaming encoder ASR for latency-critical speech applications. *arXiv preprint arXiv:2602.12241*, 2026.

[13] Somshubra Majumdar, Jagadeesh Balam, Oleksii Hrinchuk, Vitaly Lavrukhin, Vahid Noroozi, and Boris Ginsburg. Citrinet: Closing the gap between non-autoregressive and autoregressive end-to-end models for automatic speech recognition. In *arXiv preprint arXiv:2104.01721*, 2021.

[14] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proceedings of Interspeech*, pages 2613–2617, 2019.

[15] Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. Fast Conformer with linearly scalable attention for efficient speech recognition. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023.

[16] Frank Rudzicz, Aravind Kumar Namasivayam, and Talya Wolff. The TORGO database of acoustic and articulatory speech from speakers with dysarthria. *Language Resources and Evaluation*, 46:523–541, 2012.

[17] Yuan Shangguan, Jian Li, Qiao Liang, Raziel Alvarez, and Ian McGraw. Dissecting user-perceived latency of on-device E2E speech recognition. In *Proceedings of Interspeech*, pages 4553–4557, 2021.

[18] Speech Accessibility Project Challenge 2. SAPC-template: Track 2 streaming ASR starting kit. GitHub repository, 2026. Track 2 starting kit; accessed November 2, 2026.

[19] Speech Accessibility Project Challenge 2. Speech Accessibility Project Challenge 2, 2026. Accessed November 2, 2026.

[20] Kaito Takahashi, Keigo Hojo, Toshimitsu Sakai, Yukoh Wakabayashi, and Norihide Kitaoka. Fine-tuning Parakeet-TDT for dysarthric speech recognition in the Speech Accessibility Project Challenge. In *Proceedings of Interspeech 2025*, pages 3304–3308, 2025.

[21] Tianyi Tan, Xinan Chen, Xiaohuai Le, Wenzhi Fan, Xianjun Xia, Chuanzeng Huang, and Jing Lu. CBA-Whisper: Curriculum learning-based AdaLoRA fine-tuning on Whisper for low-resource dysarthric speech recognition. In *Proceedings of Interspeech 2025*, pages 3309–3313, 2025.

[22] Dominik Wagner, Ilja Baumann, Natalie Engert, Seanie Lee, Elmar Nöth, Korbinian Riedhammer, and Tobias Bocklet. Personalized fine-tuning with controllable synthetic speech from LLM-generated transcripts for dysarthric speech recognition. In *Proceedings of Interspeech 2025*, pages 3294–3298, 2025.

[23] Shiyao Wang, Jiaming Zhou, Shiwan Zhao, and Yong Qin. A self-training approach for Whisper to enhance long dysarthric speech recognition. In *Proceedings of Interspeech 2025*, pages 3299–3303, 2025.

[24] Zengwei Yao, Liyong Guo, Xiaoyu Yang, Wei Kang, Fangjun Kuang, Yifan Yang, Zengrui Jin, Long Lin, and Daniel Povey. Zipformer: A faster and better encoder for automatic speech recognition. In *International Conference on Learning Representations*, 2024.

[25] Xiuwen Zheng, Bornali Phukon, Jonghwan Na, Ed Cutrell, Kyu J. Han, Mark Hasegawa-Johnson, Pan-Pan Jiang, Aadhrik Kuila, Colin Lea, Bob MacDonald, Gautam Mantena, Venkatesh Ravichandran, Leda Sari, Katrin Tomanek, Chang D. Yoo, and Chris Zwilling. The Interspeech 2025 Speech Accessibility Project Challenge. In *Proceedings of Interspeech 2025*, pages 3269–3273, 2025.

A Cadrate ASR: Architectural and Mathematical Specifications

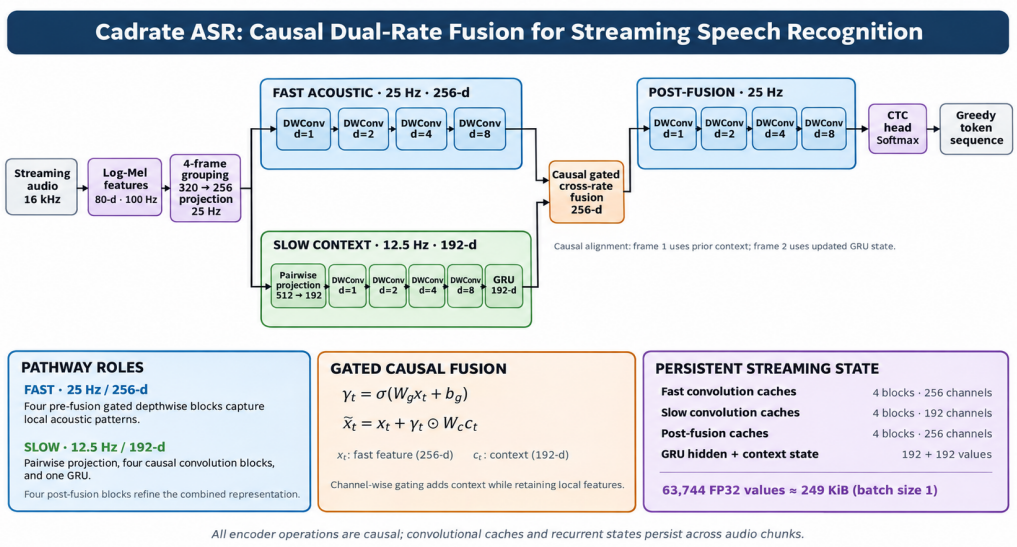


Figure 4: Detailed architecture, pathway roles, gated causal fusion equations, and persistent streaming state breakdown of Cadrate ASR.

This appendix provides the complete mathematical formulation of our proposed Cadrate ASR prototype, including its acoustic frontend, gated depthwise dilated convolutional blocks, slow recurrent contextual pathway, causal cross-rate feature fusion, CTC decoding, and streaming state memory complexity, supplementing Section 4.2.

A.1 Acoustic Frontend and Frame Grouping

The model receives mono audio sampled at 16 kHz and extracts 80-dimensional log-Mel filterbank features using a 25-ms analysis window and a 10-ms frame shift:

$$\mathbf{F} = [\mathbf{f}_1, \ldots, \mathbf{f}_N], \qquad \mathbf{f}_t \in \mathbb{R}^{80}. \tag{1}$$

To reduce the temporal rate early, four consecutive feature frames are concatenated into a 320-dimensional vector and projected into a 256-dimensional embedding space:

$$\mathbf{g}_t = [\mathbf{f}_{4t-3}; \mathbf{f}_{4t-2}; \mathbf{f}_{4t-1}; \mathbf{f}_{4t}] \in \mathbb{R}^{320}, \qquad \mathbf{x}_t = \mathrm{SiLU}(\mathrm{LN}(\mathbf{g}_t)\mathbf{W}_s + \mathbf{b}_s)\,. \tag{2}$$

This reduces the acoustic feature rate from 100 Hz to 25 Hz while preserving fine-grained intra-group frame dynamics.

A.2 Gated Depthwise Convolutional Blocks

Both the fast and slow pathways employ gated depthwise temporal convolutional blocks. For an input representation $\mathbf{x}_t$, the block applies Layer Normalization, a linear expansion, and a sigmoid gating mechanism:

$$[\mathbf{a}_t, \mathbf{b}_t] = \mathbf{W}_e\,\mathrm{LN}(\mathbf{x}_t) + \mathbf{b}_e, \qquad \mathbf{u}_t = \mathbf{a}_t \odot \sigma(\mathbf{b}_t). \tag{3}$$

A causal depthwise convolution with kernel size $k = 7$ and dilation factor $d \in \{1, 2, 4, 8\}$ is then applied, followed by a SiLU activation, pointwise projection, and residual connection:

$$\mathbf{v}_{1:T} = \mathrm{DWConv}_{k=7,d}(\mathbf{u}_{1:T}), \qquad \mathbf{y}_t = \mathbf{x}_t + \mathbf{W}_p\,\mathrm{SiLU}(\mathbf{v}_t). \tag{4}$$

A.3 Fast Acoustic Pathway, Slow Contextual Pathway, and Causal Cross-Rate Fusion

The fast acoustic pathway comprises eight convolutional blocks (two groups of four at 25 Hz). After the first four blocks, the slow pathway downsamples two consecutive fast frames to 12.5 Hz in a

192-dimensional space and processes them through four causal convolutional blocks followed by a single-layer unidirectional GRU:

$$\mathbf{s}_j^{(0)} = \mathbf{W}_d[\mathbf{x}_{2j-1}; \mathbf{x}_{2j}] + \mathbf{b}_d, \qquad \mathbf{h}_j = \mathrm{GRU}\Big(\mathbf{s}_j^{(4)}, \mathbf{h}_{j-1}\Big), \quad \mathbf{h}_j \in \mathbb{R}^{192}. \tag{5}$$

To maintain strict causality during cross-rate fusion back into the 25-Hz pathway, odd and even fast frames align with $\mathbf{c}_{2j-1} = \mathbf{h}_{j-1}$ and $\mathbf{c}_{2j} = \mathbf{h}_j$, respectively:

$$\boldsymbol{\gamma}_t = \sigma(\mathbf{W}_g \mathbf{x}_t + \mathbf{b}_g), \qquad \widetilde{\mathbf{x}}_t = \mathbf{x}_t + \boldsymbol{\gamma}_t \odot \mathbf{W}_c \mathbf{c}_t. \tag{6}$$

A.4 CTC Prediction, Streaming State Size, and Computational Complexity

The refined representations pass through a softmax layer trained with the Connectionist Temporal Classification (CTC) loss:

$$\mathbf{P}_t = \mathrm{Softmax}(\mathbf{W}_o \,\mathrm{LN}(\widetilde{\mathbf{x}}_t) + \mathbf{b}_o)\,, \qquad \mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} P(\pi_t \mid \mathbf{X}). \tag{7}$$

For a convolutional block with kernel size $k = 7$ and dilation d, the causal cache requires $L_{\mathrm{cache}} = (k-1)d$ past activations per channel. Across all 12 convolutional blocks (two 4-block fast stacks with $C_f = 256$ and one 4-block slow stack with $C_s = 192$) plus the 192-D GRU state and 192-D retained fusion state, the exact persistent state size is:

$$N_{\mathrm{state}} = 2(256)(6)(1{+}2{+}4{+}8){+}(192)(6)(1{+}2{+}4{+}8){+}192{+}192 = 63,744 \text{ values} \approx 249 \text{ KiB (FP32)}. \tag{8}$$

Ignoring constant block factors, the asymptotic time complexity scales linearly with sequence length ($T_s \approx T_f/2$):

$$\mathcal{C}_{\mathrm{Cadrate}} = \mathcal{O}\big(T_f(C_f^2 + kC_f + C_f V) + T_s(C_f C_s + C_s^2 + kC_s)\big)\,. \tag{9}$$

The convolutional modules relate to 1D separable designs in QuartzNet [11] and Citrinet [13], while our dual-rate recurrent context mechanism adapts the global context motivation of ContextNet [5] to strict fixed-state causal streaming. Table 5 provides an extended side-by-side comparison with Zipformer [24].

Table 5: Extended architectural comparison between Zipformer and Cadrate ASR.

Design Aspect	**Zipformer [24]**	**Cadrate ASR (Ours)**
Encoder organization	Multirate Transformer (6 stacks)	Two-pathway causal CNN–GRU
Temporal resolutions	50, 25, 12.5, 6.25, 12.5, and 25 Hz	25-Hz fast pathway and 12.5-Hz slow pathway
Primary temporal op.	Self-attention and nonlinear attention	Gated depthwise dilated convolution
Contextual memory	Attention-based context aggregation	Unidirectional GRU with persistent state
Normalization / Act.	BiasNorm and Swoosh activations	LayerNorm and SiLU
Inference state	Cached attention states + RNN-T predictor	Fixed 249 KiB conv/GRU caches + 1 CTC token

B Moonshine Benchmarks and Zipformer Decoding Ablations

B.1 Off-the-Shelf Moonshine and Zipformer Exploratory Benchmarks

To establish an off-the-shelf baseline prior to domain adaptation, we evaluated Moonshine v2 streaming variants [12] and an unadapted streaming Zipformer on a separate 57-recording local cohort (Tables 6 and 7). Without SAPC2 domain fine-tuning, all off-the-shelf models suffered severe recognition degradation (67.78–74.71% WER). Furthermore, while Moonshine Tiny and Small emitted initial tokens within 62–124 ms, frequent hypothesis revisions delayed their stable-first-word proxies to 2.98–4.25 s.

Table 6: Zero-shot recognition accuracy on the separate 57-recording exploratory evaluation cohort.

Model Configuration	n	WER (%)	CER (%)
Moonshine ONNX	57	67.78	46.64
Moonshine greedy	57	73.11	52.67
Off-the-shelf Zipformer	57	73.32	51.67
Moonshine Small (pre-fix)[a]	57	74.71	62.39

[a] Historical result prior to rolling-buffer correction, retained for experimental completeness.

Table 7: Exploratory streaming latency measurements (in ms, referenced to audio-send start) on 57 local recordings.

Configuration	n	First Output (ms)	Stable Proxy (ms)	TTLT (ms)
Moonshine ONNX	57	2336	2736	131
Moonshine greedy	57	2806	2825	285
Moonshine Tiny	57	62	2986	671
Moonshine Small	57	124	4252	3325
Moonshine other	57	146	4139	3824
Moonshine greedy output	57	2817	2825	306
Off-the-shelf Zipformer	57	2495	2906	546

B.2 Zipformer 15-Configuration Decoding and Padding Sweep

Table 8 details our 15-configuration local ablation on 100 development recordings evaluating numerical precision (INT8 vs. FP32), beam width (4, 6, 8, 12), blank penalties, and tail padding. INT8 4-path modified beam search reduced WER from 22.02% (greedy) to 19.47% (12.59% CER) while running $2.9\times$ faster than FP32 beam-4 (157 s vs. 457 s). Adding 1.28 s of end-of-utterance padding yielded the best overall accuracy (19.44% WER, 12.58% CER) by preventing premature truncation of prolonged dysarthric syllables.

Table 8: Local Zipformer decoding, precision, blank penalty, and tail-padding sweep on 100 development recordings.

Decoding Configuration	WER (%)	CER (%)	Total Decode Time (s)
INT8, beam 4, pad 1.28 s	**19.44**	**12.58**	277
INT8, beam 4	19.47	12.59	157
INT8, beam 4, blank 0.5	19.53	12.64	183
INT8, beam 6	19.53	12.72	1086
INT8, beam 4, blank 0.25	19.54	12.65	180
FP32, beam 4	19.64	12.82	457
FP32, beam 8	19.67	12.81	553
INT8, beam 12	19.73	12.88	1496
INT8, beam 4, pad 0.64 s	19.96	12.84	214
INT8, beam 4, blank 0.75	20.27	13.43	204
FP32, greedy	21.83	14.53	269
FP32, greedy, blank 0.5	21.92	14.60	409
INT8, greedy baseline	22.02	14.70	140
FP32, greedy, blank 1.0	22.55	15.44	2799
FP32, greedy, blank 2.0	24.83	17.72	513

C Deterministic 6-Stage Acoustic Distortion Pipeline

To evaluate acoustic hardening against severe channel and environmental distortions [6], we implemented a sequential 6-stage signal processing pipeline detailed in Section 3:

1. **Gaussian Noise:** Additive white Gaussian noise ($\sigma = 0.02$) simulating microphone pre-amp hiss.
2. **Time Stretching:** A $0.9\times$ phase-vocoder temporal stretch simulating slowed, labored syllable production.

3. **Pitch Shifting:** An upward shift of $+2$ semitones introducing vocal tract length variation.
4. **Low-Pass Filtering:** A 6th-order Butterworth filter (4000 Hz cutoff) replicating narrowband edge microphones.
5. **Discrete Echo:** A single-tap reflection (0.25 s delay, 0.4 amplitude decay).
6. **Multi-Tap Reverberation:** A simulated room impulse response with five cascading delays (0.03 s to 0.22 s) and exponentially decaying gains.

D Perceptual Speech Annotation Protocol and Rare Subgroups

Over 450 utterances from the 501-recording evaluation set were manually audited through repeated listening by the authors and categorized along non-mutually-exclusive auditory dimensions: intelligibility (*Clear*, *Unclear*, *Unintelligible*), articulation (*Slurred*, *Imprecise articulation*, *Mumbled*), fluency (*Stammered*, *Shaky*), prosody (*Slow*, *Drowsy*), and channel quality (*Muffled*, *Low volume*). Listening criteria were calibrated using dysarthric speech examples and post-stroke/neurological reference recordings; these tags represent descriptive auditory observations rather than clinical diagnoses. Table 9 reports ASR performance on the four rare perceptual categories ($n \leq 7$).

Table 9: ASR performance on low-frequency perceptual speech categories ($n \leq 7$).

Perceptual Characteristic	n	**Zipformer E19 (Ours)**		**FastConformer**		**Nemotron INT4**	
		WER	**CER**	**WER**	**CER**	**WER**	**CER**
Low volume	7	**38.24**	**34.77**	55.88	40.80	47.06	43.10
Shaky	7	**48.05**	**33.14**	75.32	49.86	75.32	67.99
Mumbled	4	**40.30**	**29.70**	79.10	52.73	79.10	71.21
Unintelligible speech	3	**66.67**	**55.89**	87.04	63.88	96.30	94.68

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction separate official Test1 results, local-only beam-search results, and the exploratory status of Cadrate ASR. Scope is limited to English SAPC2 data (Section **??**).

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section **??** covers missing component ablations, tuning on the reported development set, single-run point estimates, latency and endpointing, subjective perceptual labels, the exploratory Cadrate prototype, and data coverage.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper presents no theoretical results. Appendix A gives architectural formulations and a state-size count only.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: Sections 3 and 4 describe the data, architecture, and decoding, but the pre-trained checkpoint, synthetic-data generation procedure, and sherpa-onnx configuration are not yet fully specified.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: SAPC2 data are available under the Speech Accessibility Project data-use agreement and cannot be redistributed. [Code and checkpoints: state whether they will be released.]

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [No]

   Justification: Data splits, learning rate, batching, chunk sizes, and epoch count are reported (Sections 3–4), but the optimizer, learning-rate schedule, and how hyperparameters and decoding settings were chosen are not.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: Results are single-run point estimates. Perceptual subgroup results (Table 4) are descriptive and several subgroups have small n (Section **??**).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: Training used one NVIDIA RTX 4070 (12 GB) (Section 4.1). CPU specifications for latency measurements, per-experiment run times, and compute for preliminary experiments are not yet reported.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The authors reviewed the NeurIPS Code of Ethics. The work uses existing speech data under its data-use agreement and involves no new participant data collection.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper motivates accessibility benefits but does not yet discuss harms from misrecognition, speech-data privacy, or unequal performance across etiologies and speech clarity.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release data or models with a high risk for misuse. SAPC2 recordings are not redistributed.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: Datasets, models, and tools are cited, but licenses, versions, and terms of use for SAP, TORGO, Qwen/Kokoro TTS, icefall, sherpa-onnx, and the baselines are not yet listed.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: No new assets are released with this submission.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: Perceptual annotation was done by the authors on existing recordings (Appendix D). No crowdworkers or external participants were involved.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: We conducted no new human-subjects data collection. We use SAP recordings under the SAP data-use agreement; collection and consent were handled by the data providers.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: Qwen and Kokoro are used only as TTS systems to synthesize speech (Section 3). No LLM generated training transcripts or affected annotation or evaluation.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.